Streaming Concept-Drift Detection on a Multimodal Classification Stream: Technical Report

Dominik Zieliński

May 5, 2026

Abstract

This report describes a streaming concept-drift detection pipeline for a multimodal classification problem in which each sample is a paired image and short text caption. The pipeline synthesises a class-imbalanced data stream from an arbitrary image-folder dataset, embeds every sample on the fly with a deep visual encoder and a hashed bag-of-words text encoder, and runs streaming drift detection on top of the resulting embeddings. Two complementary monitoring regimes are considered: a *data-drift* regime that compares the embedding distribution itself, and a *performance-drift* regime that watches the prequential error of an online classifier. The report covers the pipeline end-to-end: stream construction with four drift schedules, data-only detectors based on the Fréchet distance [4], Maximum Mean Discrepancy [10], and a projected Kolmogorov–Smirnov windowing test [21]; a performance-monitoring strategy with a dual-model replacement scheme; and performance-only detectors including Adaptive Windowing [1], Page–Hinkley [20], and weighted Hoeffding-bound detection [5].

1 System overview

The pipeline processes a multimodal stream sample-by-sample. For every time step t the stream emits one image, one text caption and the corresponding class label; an embedder maps the pair to a vector $x_t \in \mathbb{R}^D$; an optional online classifier consumes (x_t, y_t) in test-then-train (*prequential*) mode [7]; and the configured drift detector is updated. Each detector update returns a record that contains, when defined, the comparison window indices, a scalar drift score, the current decision threshold, and binary drift and warning flags. The two monitoring regimes share this interface: in the data regime the detector input is the embedding itself, whereas in the performance regime the input is the per-sample classification error of an online classifier.

2 Stream construction

The stream is *semi-synthetic*: real images are drawn from a folder-based dataset (one subdirectory per class), and a short pseudo-random caption is generated for every sample so that the text modality is also non-trivial. Class identifiers are assigned in the sorted order of class names, which makes the assignment stable across datasets. Each sample is described by the tuple $(t,\ \text{image path},\ \text{label},\ \text{label index},\ \text{text},\ \text{phase})$, where the phase string records the drift regime the sample belongs to and is used only for visualisation, not by the detectors themselves.

2.1 Image stream

Image discovery is followed by a validation pass that opens each candidate image and verifies that it can be parsed, so that corrupt files are filtered out before they enter a class pool. Pools are shuffled with a deterministic random generator and each class draws images without replacement using an incrementing per-class cursor. Therefore, provided that every class pool contains enough

validated images for the requested schedule, the stream never reuses the same image. Once the drift schedule has fixed which class label provides the image at each step, the corresponding path is taken from that class pool and stored in the manifest. The actual embedding is computed lazily by the streaming pipeline at inference time, not at stream construction time.

2.2 Text stream

The textual companion of each sample is generated from a fixed vocabulary of short generic tokens (descriptive everyday words and a few stream-related terms). For each sample a length is uniformly sampled from a configurable range and the requested number of tokens is drawn from this vocabulary. A second, label-specific vocabulary contains a handful of class-aware tokens that are mixed in with a controllable probability. Setting that probability to zero produces text that is statistically independent of the label, which is the default; setting it strictly above zero injects a label-correlated signal into the text channel, allowing controlled experiments on the multimodal regime.

2.3 Multimodal embedding

The embedder supports three modalities:

- **Image only.** A pre-trained ResNet-18 [11] convolutional network is used as a fixed feature extractor: the classification head is removed and the global-average-pooled feature map is taken as the embedding (dimension 512). Inputs are pre-processed with the standard ImageNet-style protocol [3] (resize to 224×224, channel-wise normalisation). When the deep-learning framework is unavailable, the embedder falls back to a handcrafted representation that concatenates per-channel mean and standard deviation, a flattened low-resolution thumbnail, and per-channel intensity histograms.
- **Text only.** A feature-hashing vectoriser [23] maps each caption to a sparse fixed-dimensional vector without maintaining a vocabulary, with no sign cancellation and without normalisation; the result is materialised as a dense vector per sample.
- **Multimodal.** The image and text vectors are computed independently, each is ℓ_2-normalised, multiplied by a per-modality weight, concatenated, and the resulting vector is ℓ_2-normalised again. With a deep image encoder the final embedding has dimension $D = 512 + 256 = 768$ in the default configuration.

The embedder is constructed once per run and called sample-by-sample inside the streaming loop, mirroring an online deployment: there is no batching across time steps.

3 Drift schedules

Drift is injected at the *label distribution* level: at any given time t, the stream sampler picks a class label with a time-dependent probability, and the corresponding image is then drawn from that class's pool. Four drift schedules are supported. They share a common low-level mechanism, the *dominant-class* sampler:

$$P(\text{label} = \ell \mid t) = \begin{cases} p^\star, & \text{if } \ell \text{ is the dominant class at time } t, \\ \dfrac{1 - p^\star}{C - 1}, & \text{otherwise,} \end{cases} \tag{1}$$

where $p^\star \in (1/C,\ 1]$ is the dominant-class probability and C is the number of classes. Setting $p^\star = 1$ recovers pure-class segments; values strictly below 1 produce a noisy dominant-class regime that makes both the classification task and the performance-drift signal more realistic.

3.1 Abrupt drift

A single change point k splits the stream into two contiguous blocks: the pre-drift block is dominated by one class and the post-drift block by another [8]. Within each block, labels are sampled according to Equation (1) and shuffled with a per-run seed so that the post-drift block does not start with a deterministic alternation pattern.

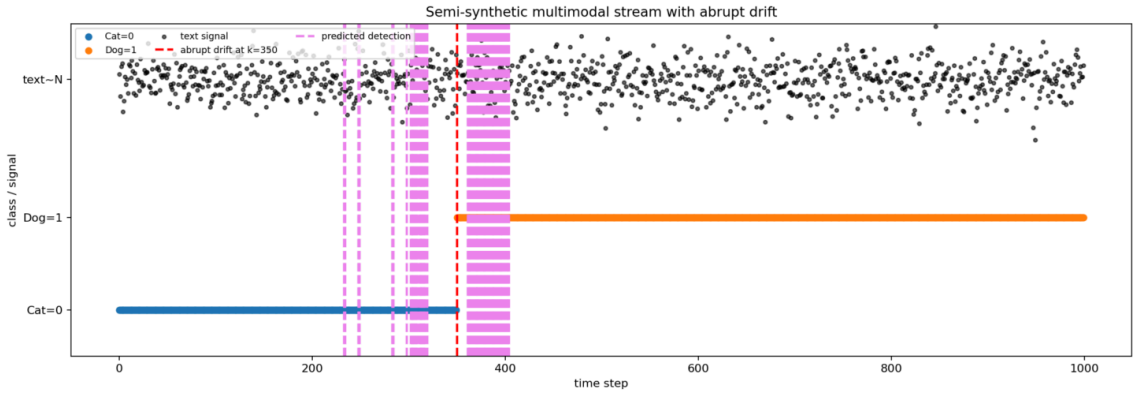

Figure 1: Abrupt drift: stream visualisation with the true change point.

3.2 Gradual drift

A linear interpolation between two label distributions runs over a closed interval $[k_{\text{start}},\ k_{\text{end}}]$, with $k_{\text{end}} > k_{\text{start}}$ [8]. Letting w_{pre} and w_{post} be the dominant-class probability vectors of the two regimes,

$$\alpha(t) = \frac{t - k_{\text{start}}}{k_{\text{end}} - k_{\text{start}}}, \qquad w(t) = (1 - \alpha(t))\, w_{\text{pre}} + \alpha(t)\, w_{\text{post}}. \tag{2}$$

Inside the gradual interval, $\alpha(t)$ is clamped to $[0, 1]$ and the label at time t is sampled from the categorical distribution $w(t)$. Outside the interval the dominant class is fixed at the corresponding endpoint distribution.

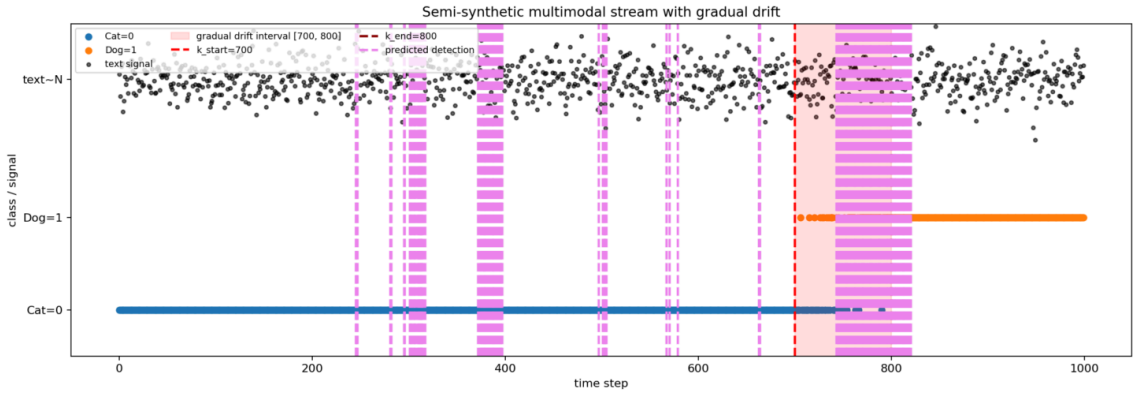

Figure 2: Gradual drift: linear ramp between two label distributions over a closed interval.

3.3 Recurrent (abrupt) drift

A strictly increasing list of change points $K = \{k_1, k_2, \ldots, k_M\}$ partitions the stream into $M + 1$ phases delimited by $\{0, k_1, \ldots, k_M, N_{\text{total}}\}$ [18]. The dominant class of phase j cycles through the

class list as $\ell_j = \text{class}_{(j \bmod C)}$, and each phase uses Equation (1) as its sampler. With $C = 2$ the schedule alternates between the two classes; with $C > 2$ it cycles through the class list in order.

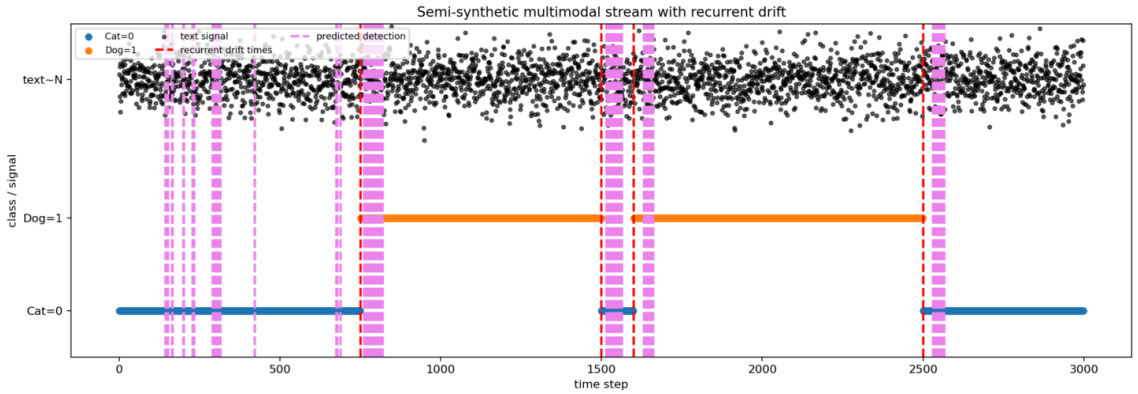

Figure 3: Recurrent (abrupt) drift: multiple change points cycling through the class list.

3.4 Recurrent gradual drift

The schedule is given as a list of non-overlapping ramp intervals $\{(k_{\text{start}}^{(i)}, k_{\text{end}}^{(i)})\}_{i=1}^{M}$, sorted by start time. The construction proceeds sequentially: a stable block precedes each ramp and is dominated by the current class; inside the i-th ramp the dominant distribution is linearly interpolated towards the next class in cyclic order, exactly as in Equation (2) but with the endpoints replaced by the current and next class distributions; once a ramp is finished, the current class is updated to the next class. After the last ramp, a final stable block uses the resulting current class up to the end of the stream. This produces a sequence of slow class flips separated by stable plateaus, a regime that is qualitatively harder than abrupt recurrence because each concept change is spread across many samples rather than marked by a sharp boundary [18].

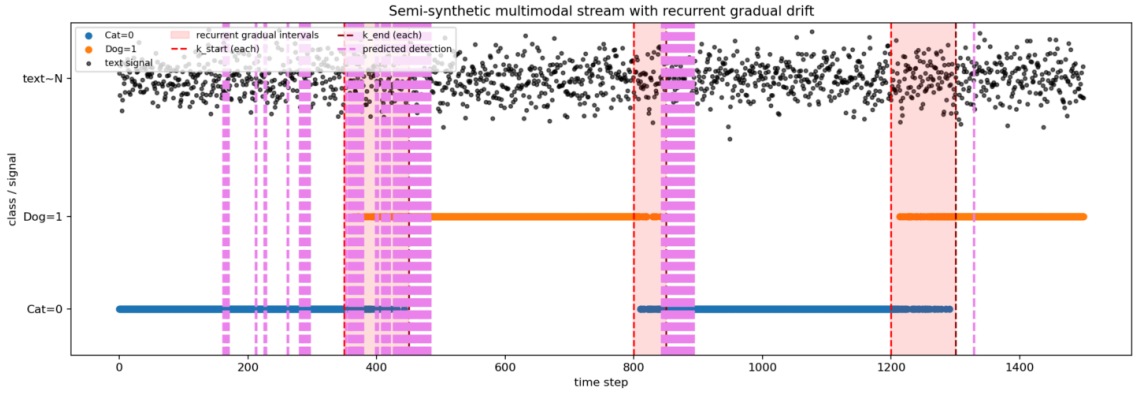

Figure 4: Recurrent gradual drift: a sequence of linear ramps cycling through the class list.

3.5 Streaming pipeline

For every sample, the pipeline (i) looks up the manifest row for time t, (ii) computes the embedding x_t, (iii) if a performance monitor is attached, runs the never-reset online classifier in test-then-train mode and feeds its binary error to the detector, (iv) updates the detector and reads back the drift and warning flags, and (v) updates the replace-on-drift classifier using the

previous step's flags so that the strategy stays causal: the detector decision computed from y_t is not allowed to affect the prediction at t itself [7].

3.6 Evaluation metrics for drift detection

Because the data stream is synthesised, the true drift onsets are known and the alarms produced by any detector can be scored against this ground truth. The evaluation protocol used in this work follows the standard streaming change-detection setting [8], in which the trade-off between detection sensitivity, detection latency and false-alarm rate is quantified by a small set of complementary metrics.

Notation. Let $K = \{k_1, k_2, \ldots, k_M\}$ denote the set of true drift onset times and $\hat{K} = \{\hat{k}_1, \hat{k}_2, \ldots, \hat{k}_R\}$ the set of times at which the detector raised an alarm, where each $\hat{k}_j$ is taken to be the right edge of the comparison window that triggered the alarm. For abrupt and recurrent abrupt drift, the onset is the single change point itself; for gradual and recurrent gradual drift, the onset is the start of the corresponding ramp. Let H denote the *detection horizon*: an alarm raised more than H steps after the true onset is no longer counted as a successful on-time detection of that event. Let T denote the index of the last sample in the stream.

Recall, precision and F_1 (horizon-based matching). A true drift event i is considered *detected within horizon* if there exists at least one alarm $\hat{k}$ that falls inside its associated match interval. For abrupt and recurrent abrupt drift, the match interval of event i is the horizon-bounded interval up to either $k_i + H$ or the index just before the next true onset, whichever comes first; for gradual and recurrent gradual drift, the match interval is the ramp interval itself, since the onset is intentionally extended in time and any alarm raised inside the ramp is on-time by construction. Letting N_h denote the number of true events that are detected within their match interval and N_a the number of alarms that fall inside any such interval, the metrics are

$$\text{Recall} = \frac{N_h}{M}, \qquad \text{Precision} = \frac{N_a}{R}, \qquad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{3}$$

In words: recall measures the fraction of true drift events that the detector caught at least once before the horizon expired; precision measures the fraction of raised alarms that landed in a legitimate event window; and F_1 is their harmonic mean. Repeated alarms inside the same gradual ramp increase precision only if they remain inside the ramp interval, but they do not increase recall beyond one detected event.

Detection delay and mean detection delay. A second matching rule, the *event-interval rule*, is used for delay-based metrics. The valid interval of event i runs from its onset k_i to the index immediately before the next true onset k_{i+1} (or to T for the last event). The first alarm that falls inside this interval is declared the valid detection $\hat{k}_i^{\text{valid}}$ of event i, and all later alarms inside the same interval are flagged as repeated false alarms (see below). The detection delay of event i is

$$\Delta_i \;=\; \hat{k}_i^{\text{valid}} - k_i, \qquad i \in \mathcal{V}, \tag{4}$$

where $\mathcal{V}$ is the set of validly detected events. The mean and median detection delays summarise the distribution of Δ_i across detected events:

$$\text{MDD} \;=\; \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \Delta_i, \qquad \text{MedDD} \;=\; \text{median}\big(\{\Delta_i\}_{i \in \mathcal{V}}\big). \tag{5}$$

In words, the mean detection delay is the average number of stream steps a detector takes to react after a true drift onset, computed over the events that were actually detected. By

construction $\Delta_i \geq 0$, since the detection cannot precede the onset under the event-interval rule. Smaller values mean a faster reaction to drift.

Missed detection rate. The missed detection rate is the complement of the detection rate under the event-interval rule:

$$\mathrm{MDR} \;=\; \frac{|K| - |\mathcal{V}|}{|K|} \;=\; 1 - \frac{|\mathcal{V}|}{M}. \tag{6}$$

In words, MDR is the fraction of true drift events for which the detector failed to raise even one alarm before the next true onset (or before the end of the stream, for the last event). A perfect detector has MDR = 0; a detector that never fires has MDR = 1. MDR differs from $1 - \mathrm{Recall}$ because it does not require the alarm to fall inside the shorter detection horizon H; it only requires an alarm inside the wider event interval.

Mean time between false alarms. Within the event-interval rule, only the first alarm inside a valid interval is credited to its event. Alarms before the first true onset, alarms outside all match intervals, and subsequent alarms inside an already detected event interval are treated as false alarms. Let $f_1 < f_2 < \cdots < f_Q$ denote their times, sorted across the stream. The mean time between false alarms is then the average inter-arrival time of false alarms,

$$\mathrm{MTFA} \;=\; \frac{1}{Q-1} \sum_{j=2}^{Q} (f_j - f_{j-1}), \qquad Q \geq 2, \tag{7}$$

and is undefined for $Q < 2$. In words, MTFA estimates how many stream steps elapse, on average, between two consecutive false alarms. Larger values mean that the detector raises spurious alarms less frequently.

Mean time ratio. The mean time ratio aggregates the previous three quantities into a single goodness measure that rewards a detector simultaneously for high detection rate, low delay and a long quiet period between false alarms:

$$\mathrm{MTR} \;=\; \frac{\mathrm{MTFA}}{\mathrm{MDD}} \cdot (1 - \mathrm{MDR}). \tag{8}$$

In words, the ratio MTFA/MDD measures how many detection delays fit inside one mean inter-false-alarm interval, that is, how *quiet* the detector is relative to how *fast* it reacts; the factor $(1 - \mathrm{MDR})$ scales this ratio down by the fraction of true drifts the detector actually catches. Higher MTR is better: a fast detector that also has rare false alarms and few misses scores high; any of slow reaction, frequent false alarms or many missed events drags the score down. MTR is undefined when MDD or MTFA cannot be evaluated, for example when there is no validly detected event or fewer than two false alarms.

4 Data-drift detectors

The detectors that watch the embedding distribution directly are Maximum Mean Discrepancy with a Gaussian kernel [10], the Fréchet distance between empirical Gaussian fits [4], and the Kolmogorov–Smirnov windowing test on a scalar projection of the embedding [21]. The first two share a common two-window framework with an adaptive, reference-free threshold; the third delegates the windowing to the underlying KSWIN test on a univariate projection.

4.1 Two-window framework with adaptive thresholds

For every new embedding, a sliding buffer of length $2W$ is updated and split into a *previous* window $X = \{x_{t-2W+1}, \ldots, x_{t-W}\}$ and a *current* window $Y = \{x_{t-W+1}, \ldots, x_t\}$. A scalar distance $s_t = d(X, Y)$ is computed and appended to the history of all observed scores. The decision threshold is taken as the empirical *quantile* of past scores excluding the current one, so the threshold is purely *adaptive* and *reference-free*: there is no fixed nominal block that defines it, the threshold is a running quantile of the same score distribution being monitored. A drift alarm is raised when the current score exceeds the upper quantile (typically the 0.95 quantile), and a softer warning is raised when the score exceeds a second, lower quantile (typically the 0.85 quantile). A configurable warm-up requires a minimum number of past scores before any decision is emitted, so that the quantile estimate is not trusted before it has stabilised.

This adaptive-quantile rule is a lightweight non-parametric control-chart heuristic: the statistic is a scale-aware distance between two adjacent windows, while the decision boundary is defined by the empirical distribution of past statistics rather than by a parametric null distribution.

4.2 Fréchet distance between empirical Gaussian fits

The Fréchet distance [4] between two Gaussian distributions with means μ_1, μ_2 and covariance matrices Σ_1, Σ_2 is

$$d^2(\mathcal{N}_1, \mathcal{N}_2) = \|\mu_1 - \mu_2\|_2^2 + \operatorname{tr}\Big(\Sigma_1 + \Sigma_2 - 2\,(\Sigma_1\Sigma_2)^{1/2}\Big)\,. \tag{9}$$

This is the same Gaussian-feature distance that underlies the Fréchet Inception Distance used to compare deep-feature distributions [12]. A direct evaluation of Equation (9) requires the matrix square root of a $D \times D$ matrix, whose principal-branch computation costs $\mathcal{O}(D^3)$ and is typically realised via a Schur decomposition [13]. For the embedding sizes used here ($D \in \{256,\ 512,\ 768\}$) and the small sliding-window sizes typical of streaming ($n = m = W$, with W on the order of 50), both empirical covariances are heavily rank-deficient, since $\operatorname{rank}(\Sigma) \le \min(n,\ D) - 1$. The textbook formulation is therefore both expensive and numerically fragile in this regime.

The implementation evaluates the same quantity entirely in the *sample space*. Let $\bar{X}, \bar{Y}$ be the empirical means and define the centred, square-root-normalised data matrices

$$U = \frac{(X - \bar{X})^\top}{\sqrt{n-1}} \in \mathbb{R}^{D \times n}, \qquad V = \frac{(Y - \bar{Y})^\top}{\sqrt{m-1}} \in \mathbb{R}^{D \times m},$$

so that $\Sigma_1 = UU^\top$ and $\Sigma_2 = VV^\top$. The trace of the matrix square root then collapses, via the singular-value decomposition of the cross matrix $U^\top V \in \mathbb{R}^{n \times m}$, to

$$\operatorname{tr}\Big((\Sigma_1\Sigma_2)^{1/2}\Big) = \operatorname{tr}\Big((UU^\top VV^\top)^{1/2}\Big) = \sum_i \sigma_i(U^\top V), \tag{10}$$

while the individual covariance traces simplify to the squared Frobenius norms $\|U\|_F^2$ and $\|V\|_F^2$. The reported score is $\widehat{d}^2 = \|\bar{X} - \bar{Y}\|_2^2 + \|U\|_F^2 + \|V\|_F^2 - 2\sum_i \sigma_i(U^\top V)$, clamped to be non-negative to absorb floating-point round-off. The dominant cost is the singular-value decomposition of an $n \times m$ matrix, namely $\mathcal{O}(\min(n,m)^2 \max(n,m))$ and $\mathcal{O}(W^3)$ when $n = m = W$, rather than $\mathcal{O}(D^3)$. This is several orders of magnitude cheaper for the dimensions used here and is also amenable to GPU acceleration.

The score responds to changes in either the mean of the embedding distribution or its second-order structure (the singular spectrum of the covariance), which makes it a useful one-number summary for visual-feature drift, where both first- and second-order shifts are common.

4.3 Maximum Mean Discrepancy with a Gaussian kernel

The squared Maximum Mean Discrepancy [10] between two samples $X = \{x_1, \ldots, x_n\}$ and $Y = \{y_1, \ldots, y_m\}$ with respect to a positive-definite kernel $k(\cdot, \cdot)$ admits the unbiased empirical estimator

$$\widehat{\mathrm{MMD}}_u^2(X, Y) = \frac{1}{n(n-1)} \sum_{i \neq j} k(x_i, x_j) + \frac{1}{m(m-1)} \sum_{i \neq j} k(y_i, y_j) - \frac{2}{nm} \sum_{i,j} k(x_i, y_j). \tag{11}$$

The implementation uses the Gaussian RBF kernel $k(x, y) = \exp(-\gamma \|x - y\|_2^2)$. The two within-group sums in Equation (11) are realised by zeroing the diagonal of the corresponding kernel matrix and dividing by $n(n-1)$ rather than n^2, which yields the unbiased estimator without the diagonal self-similarities. The cross term is simply the mean of the cross kernel matrix.

The kernel bandwidth γ is set per call by the *median heuristic* [9]: pairwise squared distances are computed on the union of the two windows (sub-sampled to a fixed cap with a deterministic seed for stability), the strict upper triangle is extracted, non-positive entries are dropped, and $\gamma = 1/\text{median}$ of the surviving squared distances. This adapts the kernel to the local scale of the embedding distribution at every comparison, which matters because the two-window framework feeds the empirical distribution of past scores back into the threshold: a slowly changing scale would otherwise inflate the threshold quantile and reduce sensitivity. Optionally, the bandwidth can be fixed in advance to disable the heuristic, although the default behaviour is data-driven.

The unbiased finite-sample estimate in Equation (11) can be slightly negative because the diagonal self-similarities are omitted. The implementation clips the reported score to zero for plotting and thresholding. High values indicate that the kernel mean embeddings of the two windows are far apart in the reproducing-kernel Hilbert space induced by the Gaussian kernel [10], which captures differences in the underlying distributions and not only differences in the mean.

4.4 Kolmogorov–Smirnov windowing on a scalar projection

The third data-mode detector reduces each embedding to a scalar and feeds the resulting univariate stream into a sliding Kolmogorov–Smirnov windowing test [21]. Three projections are available:

- the ℓ_2 norm of the embedding, sensitive only to magnitude;
- the mean of the embedding components, a generic linear projection;
- the ℓ_2 distance from a running exponentially weighted moving average of the embedding,

  $$z_t = \|x_t - c_{t-1}\|_2, \qquad c_t = (1 - \alpha)\, c_{t-1} + \alpha\, x_t,$$

  initialised lazily with the first incoming embedding. This default projection captures a *distance-from-recent-typical* signal: when the underlying class distribution shifts, the embedding stops agreeing with the running centroid and the projected scalar grows.

The univariate stream is then fed into the Kolmogorov–Smirnov windowing test. The detector keeps a sliding window over the projected scalar; at each new sample, the most recent sub-window is compared against a random sub-sample of the older portion of the sliding window using the two-sample Kolmogorov–Smirnov statistic [17], and a drift is declared when the test rejects the null hypothesis of equal distributions at the configured significance level. The detector exposes a one-sided score by reporting one minus the test p-value, so that the drift-score plot has a horizontal cut-off corresponding to the statistical decision boundary. The Kolmogorov–Smirnov windowing test does not emit warnings, so the replace-on-drift strategy in the performance regime falls back to a cold model when this detector fires (Section 5).

5 Performance-monitoring strategy

In the performance regime the detector input is no longer the multivariate embedding but the binary per-sample classification error of an online classifier; the detector's task is to spot a change in the prequential error process [7].

5.1 Underlying online classifier

The shared base classifier is a streaming pipeline that combines a running standardiser with a multi-class logistic regressor:

- The standardiser updates a per-coordinate running mean and variance using Welford's online algorithm [24] and rescales every incoming embedding to zero mean and unit variance. This keeps the linear classifier well-conditioned online, since the deep visual features and the hashed text features are not zero-mean unit-variance: the visual features have heavy-tailed coordinates and the hashed text vector is sparse and non-negative.
- The logistic regressor is wrapped in a one-vs-rest meta-estimator [22] so that the same code path supports both binary and multi-class streams. Each binary sub-classifier is updated online with stochastic gradient descent on the log-loss, which makes the whole pipeline a streaming counterpart of a standard linear classifier [2].
- Both stages are updated and queried sample-by-sample through the `river` streaming machine-learning framework [19].

The pipeline is scored *prequentially*: for every sample the monitor first predicts (test) and only then updates with the true label (train), following the standard prequential evaluation protocol for streaming classifiers [7]. This is the only training source: there is no held-out fit phase, so the running accuracy reported by the monitor is precisely the prequential accuracy on the synthetic stream.

5.2 Dual-model replacement strategy

The performance monitor maintains two parallel pipelines on the same stream:

- **Never-reset model.** A single online classifier learns from every sample regardless of detector flags. Its per-sample error is the natural detector input, because its trajectory is unaffected by the detector's own decisions: the detector never reads back a signal that depends on its own past decisions, which would otherwise create a feedback loop. The model is never replaced, so this strategy represents the behaviour of a classical online learner that trusts the stream and relies on plasticity alone.
- **Replace-on-drift model with a warned shadow.** The same classifier class is run in parallel, but is reactive to the detector flags. On a *warning* flag a fresh shadow model is instantiated (if none exists) and starts learning from every subsequent sample in parallel with the active model. On a *drift* flag, the active model is discarded and the shadow is promoted in its place, then the shadow is cleared. This warm-shadow scheme follows the warning-then-drift design used by classical drift detectors such as DDM [6]. When a detector that does not emit warnings fires drift directly, the monitor falls back to a cold model with no warm-up, which is the only honest option in that regime.

For every sample, the pipeline records the prediction, error and running accuracy of each of the two strategies, the cumulative number of model replacements, and the per-sample wall-clock costs of prediction and update for the active model and the shadow separately. The cost of training the shadow is reported as part of the replace-on-drift strategy, because that cost is intrinsic to it: without a hot shadow, the swap on drift would promote a cold model.

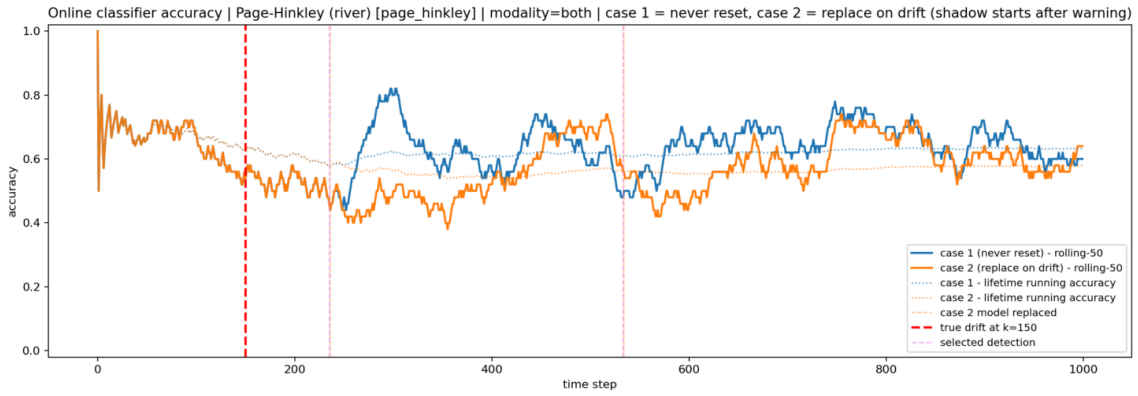

Figure 5: Prequential accuracy of the never-reset and replace-on-drift strategies, with markers at each model replacement.

6 Performance-drift detectors

In the performance regime, the detector input collapses to a single binary value per step. Four detectors are available; each is briefly described below, together with the parameter that governs the alarm rate.

6.1 Kolmogorov–Smirnov windowing

This is the same Kolmogorov–Smirnov windowing test as in Section 4.4 [21]; only the input changes. The test compares a recent sub-window of the binary error signal to a random subsample of the older portion of the sliding window using the two-sample Kolmogorov–Smirnov statistic [17], and fires drift when the test rejects the null at the configured significance level. The significance level is the main parameter governing the alarm rate: smaller values mean stricter rejection (fewer false alarms but more delay), larger values mean more sensitive detection (faster but noisier).

6.2 Adaptive Windowing

The Adaptive Windowing detector (ADWIN) [1] maintains a variable-length window over a univariate stream statistic and tests whether the older and newer parts of the window appear to come from the same distribution. In our experiments we used the implementation provided by `river.drift.ADWIN`. Thus, the detector was not implemented by explicitly evaluating a separate cut-threshold formula; at each step, the monitored statistic was passed to the detector via `update`, and a drift event was registered whenever River's built-in flag `drift_detected` became true.

ADWIN adapts its window automatically: during stable periods the window can grow, while after a detected change the older part of the window is discarded so that subsequent estimates are based on more recent samples. The main sensitivity parameter is the significance value δ. Smaller values of δ make the detector more conservative, typically reducing false alarms at the cost of longer detection delay, whereas larger values make the detector more reactive but potentially noisier. Unless stated otherwise, the remaining implementation parameters were left at River's defaults.

6.3 Page–Hinkley test

The Page–Hinkley test [20] is a CUSUM-style sequential test for changes in the mean of a stream. It maintains a running estimate of the mean, a cumulative sum of the centred residuals (with a small slack constant), and fires a drift alarm when the excursion of the cumulative statistic

exceeds a fixed threshold. The main parameter is the threshold magnitude: the smaller the threshold, the more sensitive the detector. The detector does not emit warnings.

6.4 Hoeffding-bound drift detection (weighted)

The weighted Hoeffding-bound drift detector [5] is designed for binary error streams, which is exactly the signal produced by the never-reset prequential classifier. It maintains an exponentially weighted estimate of the error rate and uses Hoeffding's inequality [14] to test whether the current weighted mean is significantly higher than the lowest weighted mean seen so far in the stable phase. Two confidence levels control two thresholds: warnings fire when the looser bound is exceeded and drifts fire when the stricter bound is exceeded. The main parameter governing the alarm rate is the drift confidence (the alpha for the drift bound); the warning confidence is set larger so that the warning threshold is crossed before the drift threshold, which is the regime the warm-shadow strategy exploits to warm-start the replacement classifier (Section 5).

7 Benchmarking results

This section reports the first benchmark results obtained with the attached benchmarking and visualisation notebooks. Two binary image datasets were used: Cats vs Dogs [15] and Real vs Fake Faces [16]. In both cases the stream was built from real image folders, paired with the same pseudo-random caption generator described above, embedded in the multimodal setting, and evaluated under the recurrent-gradual drift schedule. The benchmark driver ran each compatible combination of monitoring target and detector for ten random-seed trials. In each trial, both the seed and the recurrent-gradual ramp intervals were sampled anew; the script then wrote one granular row per trial and an aggregated row with means, standard deviations and counts for each (drift target, detector, drift type, modality) combination. The notebooks loaded these granular and aggregated CSV files, separated the data-drift and performance-drift regimes, and plotted detection quality, detection delay and computational cost. Each notebook contained 70 granular runs and 7 aggregated groups: three data-target detectors (MMD, Fréchet and KSWIN) and four performance-target detectors (ADWIN, HDDM-W, KSWIN and Page–Hinkley), all with `modality=both` and `drift_type=gradual_recurrent`.

Table 1 summarises the data-drift detectors. On Cats vs Dogs, MMD achieved the highest recall (0.97) and essentially tied Fréchet distance on F_1; Fréchet distance gave the best precision and a similar mean delay. On Real vs Fake Faces, MMD again gave the best F_1 and the shortest mean delay, while Fréchet distance had the best precision. KSWIN was computationally cheaper in both datasets, but it lost recall and F_1 relative to the two multivariate two-window detectors. The large alarm counts for MMD and Fréchet explain why their precision is much lower than their recall: they detect many genuine drifts, but also fire repeatedly inside and around the same recurrent-gradual transitions.

Table 1: Data-target benchmark results for recurrent-gradual drift. Values are mean ± standard deviation over ten trials, except alarms and per-sample time, which are shown as means. Delay is measured in stream samples.

Dataset	Detector	Recall	Precision	F_1	Delay	Alarms/trial	ms/sample
Cats vs Dogs	Fréchet	0.85 ± 0.20	0.43 ± 0.22	0.53 ± 0.21	52.1 ± 42.7	87.4	5.03
Cats vs Dogs	MMD	0.97 ± 0.11	0.39 ± 0.17	0.53 ± 0.17	53.2 ± 26.2	63.7	39.72
Cats vs Dogs	KSWIN	0.75 ± 0.23	0.30 ± 0.09	0.42 ± 0.12	99.9 ± 56.1	7.0	3.36
Real vs Fake Faces	Fréchet	0.73 ± 0.25	0.41 ± 0.25	0.48 ± 0.22	71.6 ± 40.7	56.3	4.11
Real vs Fake Faces	MMD	0.73 ± 0.37	0.40 ± 0.22	0.51 ± 0.28	57.2 ± 42.9	56.1	39.03
Real vs Fake Faces	KSWIN	0.65 ± 0.31	0.35 ± 0.19	0.44 ± 0.22	100.5 ± 86.4	6.3	3.77

Table 2 summarises the performance-drift regime, where detectors observe only the binary

prequential error of the never-reset classifier. These results are much weaker than the data-drift results. For Cats vs Dogs, the classifier accuracy was about 0.968 in both classifier strategies, leaving almost no informative error-rate shift for the performance detectors; all horizon-based recalls and F_1 scores were zero. For Real vs Fake Faces, the classifier accuracy was lower, about 0.616 for the never-reset model, and the error stream contained a slightly stronger signal. Even there, however, the best F_1 was only 0.13 and ADWIN raised no alarms. This supports the main experimental conclusion from these two notebooks: under the current semi-synthetic label-distribution drifts, monitoring the embedding distribution is more informative than monitoring the classification error alone. The delay column is undefined when no valid event-interval detection exists; a detector may still have a listed delay together with zero horizon recall when its first credited event-interval alarm occurs after the stricter horizon-based match window.

Table 2: Performance-target benchmark results for recurrent-gradual drift. Values are mean ± standard deviation over ten trials where a standard deviation is defined. Case 1 is the never-reset classifier and Case 2 is the replace-on-drift classifier.

Dataset	Detector	Recall	Precision	F_1	Delay	Alarms/trial	Case 1 acc.	Case 2 acc.
Cats vs Dogs	ADWIN	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	—	0.0	0.968	0.968
Cats vs Dogs	HDDM-W	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	—	0.0	0.968	0.968
Cats vs Dogs	KSWIN	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	—	0.0	0.968	0.968
Cats vs Dogs	Page–Hinkley	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	141.0	0.1	0.968	0.967
Real vs Fake Faces	ADWIN	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	—	0.0	0.616	0.616
Real vs Fake Faces	HDDM-W	0.12 ± 0.25	0.15 ± 0.34	0.13 ± 0.28	180.9 ± 70.1	1.3	0.616	0.604
Real vs Fake Faces	KSWIN	0.10 ± 0.21	0.10 ± 0.21	0.10 ± 0.21	209.4 ± 87.4	1.2	0.616	0.605
Real vs Fake Faces	Page–Hinkley	0.10 ± 0.21	0.20 ± 0.42	0.13 ± 0.28	155.5 ± 125.9	0.6	0.616	0.612

8 Conclusion

This report described an end-to-end streaming concept-drift benchmark on a multimodal classification stream: image-folder ingestion with deep visual features and hashed bag-of-words text features combined into an ℓ_2-normalised multimodal vector; four drift schedules (abrupt, gradual, recurrent, recurrent gradual); two complementary monitoring regimes; and a dual prequential classifier [7] that feeds either KSWIN [21], Adaptive Windowing [1], Page–Hinkley [20], or weighted Hoeffding-bound detection [5]. The data-only regime uses two adaptive-quantile sliding-window detectors based on Maximum Mean Discrepancy [10] and the Fréchet distance [4], plus a projected KSWIN test [21]. The two classifier-side strategies are run in parallel: one is never reset, and one is replaced on drift with a warned shadow. They are reported with separate accuracy and timing figures, so that the trade-off between plasticity-only and reset-on-drift learning can be measured on the same stream.

References

[1] Albert Bifet and Ricard Gavaldà. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*, pages 443–448, 2007.

[2] Léon Bottou. Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010*, pages 177–186. Physica-Verlag, 2010.

[3] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.

[4] D. C. Dowson and B. V. Landau. The Fréchet distance between multivariate normal distributions. *Journal of Multivariate Analysis*, 12(3):450–455, 1982.

[5] Isvani Frías-Blanco, José del Campo-Ávila, Gonzalo Ramos-Jiménez, Rafael Morales-Bueno, Agustín Ortiz-Díaz, and Yailé Caballero-Mota. Online and non-parametric drift detection methods based on Hoeffding's bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3):810–823, 2015.

[6] João Gama, Pedro Medas, Gladys Castillo, and Pedro Pereira Rodrigues. Learning with drift detection. In *Brazilian Symposium on Artificial Intelligence (SBIA)*. Springer, 2004.

[7] João Gama, Raquel Sebastião, and Pedro Pereira Rodrigues. On evaluating stream learning algorithms. *Machine Learning*, 90(3):317–346, 2013.

[8] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):44:1–44:37, 2014.

[9] Damien Garreau, Wittawat Jitkrittum, and Motonobu Kanagawa. Large sample analysis of the median heuristic. *arXiv preprint arXiv:1707.07269*, 2017.

[10] Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13(25):723–773, 2012.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[12] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In *Advances in Neural Information Processing Systems*, pages 6626–6637, 2017.

[13] Nicholas J. Higham. *Functions of Matrices: Theory and Computation*. Society for Industrial and Applied Mathematics, 2008.

[14] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[15] KaraKaggle. Kaggle Cat vs Dog Dataset. https://www.kaggle.com/datasets/karakaggle/kaggle-cat-vs-dog-dataset. Accessed: 2026-04-29.

[16] Troy Kueh. Real vs Fake Faces – StyleGAN3. https://www.kaggle.com/datasets/troykueh/real-vs-fake-faces-stylegan3. Accessed: 2026-04-29.

[17] Frank J. Massey. The Kolmogorov–Smirnov test for goodness of fit. *Journal of the American Statistical Association*, 46(253):68–78, 1951.

[18] Leandro L. Minku and Xin Yao. DDD: A new ensemble approach for dealing with concept drift. *IEEE Transactions on Knowledge and Data Engineering*, 24(4):619–633, 2012.

[19] Jacob Montiel, Max Halford, Saulo Martiello Mastelini, Geoffrey Bolmier, Raphael Sourty, Robin Vaysse, Adil Zouitine, Heitor Murilo Gomes, Jesse Read, Talel Abdessalem, and Albert Bifet. River: Machine learning for streaming data in Python. *Journal of Machine Learning Research*, 22(110):1–8, 2021.

[20] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.

[21] Christoph Raab, Moritz Heusinger, and Frank-Michael Schleif. Reactive soft prototype computing for concept drift streams. *Neurocomputing*, 416:340–351, 2020.

[22] Ryan Rifkin and Aldebaro Klautau. In defense of one-vs-all classification. *Journal of Machine Learning Research*, 5:101–141, 2004.

[23] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, pages 1113–1120, 2009.

[24] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962.